SeniorShield Hearing Aid Connectivity Specification

Overview

Hearing aid connectivity is a **critical accessibility feature** for SeniorShield. Since 65% of seniors 70+ have hearing loss, direct integration with hearing aids dramatically improves the user experience by:

1. Providing clear audio directly to hearing aids
2. Reducing feedback and echo
3. Eliminating need for external speakers
4. Allowing volume control independent of phone
5. Supporting multiple hearing aid brands
6. Enabling seamless switching between audio sources

---

1. Market Context & Statistics

Hearing Loss in Seniors

Plain Text

```
PREVALENCE:
- 65% of seniors 70+ have hearing loss
- 35% of seniors 65-74 have hearing loss
- 50% of seniors 75+ have hearing loss
- Total addressable market: ~30 million seniors in US

HEARING AID ADOPTION:
- Only 30% of seniors with hearing loss use hearing aids
- 70% of seniors with hearing loss don't use hearing aids
- Cost is primary barrier ($2,000-6,000 per pair)
- Stigma is secondary barrier
- Technology compatibility is tertiary barrier

MARKET OPPORTUNITY:
- 21 million seniors with untreated hearing loss
- If 10% adopt SeniorShield with hearing aid support: 2.1M users
- At $12.99/month: $326M annual revenue opportunity
```

Supported Hearing Aid Brands

Plain Text

```
MAJOR BRANDS (85% market share):
1. ReSound (GN Store Nord) — 25% market share
   - Models: Linx, Quattro, Omnia, One
   - Connectivity: Bluetooth, 2.4GHz proprietary
   - iOS: MFi (Made for iPhone)
   - Android: ASHA (Android Hearing Aid Audio Service)

2. Phonak (Sonova) — 20% market share
   - Models: Audeo, Bolero, Naida, Virto
   - Connectivity: Bluetooth, proprietary
   - iOS: MFi (Made for iPhone)
   - Android: ASHA

3. Widex (Sonova) — 18% market share
   - Models: Moment, Beyond, Evoke
   - Connectivity: Bluetooth, proprietary
   - iOS: MFi (Made for iPhone)
   - Android: ASHA

4. Signia (WS Audiology) — 15% market share
   - Models: Styletto, Silk, Pure
   - Connectivity: Bluetooth, proprietary
   - iOS: MFi (Made for iPhone)
   - Android: ASHA

5. Oticon (Demant) — 10% market share
   - Models: More, Intent, Opn
   - Connectivity: Bluetooth, proprietary
   - iOS: MFi (Made for iPhone)
   - Android: ASHA

6. Unitron (Sonova) — 5% market share
   - Models: Moxi, Stride, Max
   - Connectivity: Bluetooth, proprietary
   - iOS: MFi (Made for iPhone)
   - Android: ASHA

7. Starkey — 4% market share
   - Models: Livio, Muse, Evolv
   - Connectivity: Bluetooth, proprietary
   - iOS: MFi (Made for iPhone)
   - Android: ASHA

8. Bernafon (Sonova) — 3% market share
```

```
    - Models: Viron, Sonate
    - Connectivity: Bluetooth, proprietary
    - iOS: MFi (Made for iPhone)
    - Android: ASHA
```

---

2. iOS Hearing Aid Connectivity

MFi (Made for iPhone) Implementation

Plain Text

```
WHAT IS MFi?
- Apple's official program for hearing aid manufacturers
- Provides direct audio streaming to hearing aids
- Requires hearing aids to be MFi-certified
- Works with iPhone 5S and later
- Works with iOS 12.2 and later

SUPPORTED MODELS:
- ReSound: Linx, Quattro, Omnia, One
- Phonak: Audeo, Bolero, Naida, Virto
- Widex: Moment, Beyond, Evoke
- Signia: Styletto, Silk, Pure
- Oticon: More, Intent, Opn
- Unitron: Moxi, Stride, Max
- Starkey: Livio, Muse, Evolv
- Bernafon: Viron, Sonate

IMPLEMENTATION STEPS:

1. ADD HEARING AID SUPPORT TO INFO.PLIST
```xml
<key>UIRequiredDeviceCapabilities</key>
<array>
  <string>bluetooth-le</string>
  <string>audio-session</string>
</array>

<key>NSBluetoothPeripheralUsageDescription</key>
<string>SeniorShield needs access to Bluetooth to connect to your hearing aids

<key>NSBluetoothCentralUsageDescription</key>
<string>SeniorShield needs access to Bluetooth to find and connect to your hear
```
```

1. IMPORT FRAMEWORKS

Swift

```swift
import AVFoundation
import CoreBluetooth
import MediaPlayer
```

1. REQUEST USER PERMISSION

Swift

```swift
// Request Bluetooth permission
CBCentralManager.requestAuthorization()

// Request audio permission
AVAudioSession.sharedInstance().requestRecordPermission { granted in
    if granted {
        // User granted audio permission
    }
}
```

1. DISCOVER HEARING AIDS

Swift

```swift
import CoreBluetooth

class HearingAidManager: NSObject, CBCentralManagerDelegate {
    var centralManager: CBCentralManager?
    var discoveredHearingAids: [CBPeripheral] = []

    override init() {
        super.init()
        centralManager = CBCentralManager(delegate: self, queue: nil)
    }

    func startScanning() {
        // Scan for hearing aid services
        let hearingAidServiceUUIDs = [
            CBUUID(string: "FDF0"),  // Hearing Aid Audio Service
            CBUUID(string: "180D"),  // Heart Rate Service (some models)
        ]

        centralManager?.scanForPeripherals(
            withServices: hearingAidServiceUUIDs,
            options: [CBCentralManagerScanOptionAllowDuplicatesKey: false]
        )
```

```
    }

    func centralManager(
        _ central: CBCentralManager,
        didDiscover peripheral: CBPeripheral,
        advertisementData: [String : Any],
        rssi RSSI: NSNumber
    ) {
        // Found a hearing aid
        discoveredHearingAids.append(peripheral)

        // Display to user with signal strength
        let signalStrength = calculateSignalStrength(rssi: RSSI)
        print("Found hearing aid: \(peripheral.name ?? "Unknown") - Signal: \(s
    }

    func calculateSignalStrength(rssi: NSNumber) -> Int {
        let rssiValue = rssi.intValue
        let maxRSSI: Int = -30
        let minRSSI: Int = -100

        let strength = max(0, min(100, (rssiValue - minRSSI) * 100 / (maxRSSI -
        return strength
    }
}
```

1. CONNECT TO HEARING AID

Swift

```
func connectToHearingAid(_ peripheral: CBPeripheral) {
    // Stop scanning
    centralManager?.stopScan()

    // Connect to selected hearing aid
    centralManager?.connect(peripheral, options: nil)
}

func centralManager(
    _ central: CBCentralManager,
    didConnect peripheral: CBPeripheral
) {
    // Successfully connected
    print("Connected to hearing aid: \(peripheral.name ?? "Unknown")")

    // Discover services
    peripheral.delegate = self
```

```
    peripheral.discoverServices([CBUUID(string: "FDF0")])
}
```

1. CONFIGURE AUDIO ROUTING

Swift

```swift
import AVFoundation

func configureAudioRouting(toHearingAid: Bool) {
    let audioSession = AVAudioSession.sharedInstance()

    do {
        if toHearingAid {
            // Route audio to hearing aid
            try audioSession.setCategory(
                .playAndRecord,
                mode: .default,
                options: [.duckOthers, .defaultToSpeaker]
            )

            // Enable hearing aid audio
            try audioSession.setPreferredInput(nil) // Use default (hearing aid

        } else {
            // Route audio to phone speaker
            try audioSession.setCategory(
                .playAndRecord,
                mode: .default,
                options: [.duckOthers, .defaultToSpeaker]
            )
        }

        try audioSession.setActive(true, options: .notifyOthersOnDeactivation)

    } catch {
        print("Error configuring audio: \(error.localizedDescription)")
    }
}
```

1. IMPLEMENT VOLUME CONTROL

Swift

```swift
class HearingAidVolumeController {
    var phoneVolume: Float = 0.5
    var hearingAidVolume: Float = 0.5
```

```swift
    func setPhoneVolume(_ volume: Float) {
        phoneVolume = max(0, min(1, volume))

        let audioSession = AVAudioSession.sharedInstance()
        do {
            try audioSession.setActive(true)
            // Volume is controlled via system volume for phone speaker
        } catch {
            print("Error setting phone volume: \(error.localizedDescription)")
        }
    }

    func setHearingAidVolume(_ volume: Float) {
        hearingAidVolume = max(0, min(1, volume))

        // Send volume command to hearing aid via Bluetooth
        // Implementation depends on specific hearing aid model
    }

    func getPhoneVolume() -> Float {
        return phoneVolume
    }

    func getHearingAidVolume() -> Float {
        return hearingAidVolume
    }
}
```

1. HANDLE AUDIO INTERRUPTIONS

Swift

```swift
func setupAudioInterruptionHandling() {
    NotificationCenter.default.addObserver(
        self,
        selector: #selector(handleAudioInterruption),
        name: AVAudioSession.interruptionNotification,
        object: AVAudioSession.sharedInstance()
    )
}

@objc func handleAudioInterruption(notification: Notification) {
    guard let userInfo = notification.userInfo,
          let typeValue = userInfo[AVAudioSessionInterruptionTypeKey] as? UInt,
          let type = AVAudioSession.InterruptionType(rawValue: typeValue) else
        return
    }
```

```
    if type == .began {
        // Audio was interrupted (e.g., phone call)
        print("Audio interrupted")
    } else if type == .ended {
        // Audio interruption ended
        if let optionsValue = userInfo[AVAudioSessionInterruptionOptionKey] as?
            let options = AVAudioSession.InterruptionOptions(rawValue: optionsV
            if options.contains(.shouldResume) {
                // Resume audio playback
                print("Resuming audio")
            }
        }
    }
}
```

1. FEEDBACK REDUCTION

Swift

```
func enableFeedbackReduction() {
    let audioSession = AVAudioSession.sharedInstance()

    do {
        // Enable automatic gain control
        try audioSession.setCategory(
            .playAndRecord,
            mode: .default,
            options: [.duckOthers, .defaultToSpeaker]
        )

        // Reduce echo and feedback
        try audioSession.setPreferredSampleRate(44100)
        try audioSession.setPreferredIOBufferDuration(0.01)

    } catch {
        print("Error enabling feedback reduction: \(error.localizedDescription)
    }
}
```

1. TEST CONNECTION

Swift

```
func testHearingAidConnection() {
    // Play test tone to hearing aid
    let audioEngine = AVAudioEngine()
```

```
    let audioSession = AVAudioSession.sharedInstance()

    do {
        try audioSession.setActive(true)

        // Generate test tone (440 Hz, 1 second)
        let frequency: Float = 440.0
        let sampleRate: Float = 44100.0
        let duration: Float = 1.0
        let frameCount = Int(sampleRate * duration)

        let audioBuffer = AVAudioPCMBuffer(
            pcmFormat: AVAudioFormat(standardFormatWithSampleRate: Double(sampl
            frameCapacity: AVAudioFrameCount(frameCount)
        )!

        audioBuffer.frameLength = AVAudioFrameCount(frameCount)

        let floatData = audioBuffer.floatChannelData![0]
        for i in 0..<frameCount {
            let phase = Float(i) / sampleRate * frequency * 2.0 * .pi
            floatData[i] = sin(phase) * 0.3 // 30% volume
        }

        // Play test tone
        audioEngine.attachNode(AVAudioPlayerNode())
        audioEngine.start()

        print("Test tone sent to hearing aid")

    } catch {
        print("Error testing hearing aid connection: \(error.localizedDescripti
    }
}
```

3. Android Hearing Aid Connectivity

ASHA (Android Hearing Aid Audio Service) Implementation

Plain Text

```
WHAT IS ASHA?
- Android's official hearing aid audio service (API 29+)
- Provides direct audio streaming to hearing aids
- Requires hearing aids to be ASHA-certified
```

```
- Works with Android 10 and later
- Uses Bluetooth Low Energy (BLE)

SUPPORTED MODELS:
- Same brands as iOS (ReSound, Phonak, Widex, Signia, Oticon, etc.)

IMPLEMENTATION STEPS:

1. ADD PERMISSIONS TO ANDROIDMANIFEST.XML
```xml
<uses-permission android:name="android.permission.BLUETOOTH" />
<uses-permission android:name="android.permission.BLUETOOTH_ADMIN" />
<uses-permission android:name="android.permission.BLUETOOTH_SCAN" />
<uses-permission android:name="android.permission.BLUETOOTH_CONNECT" />
<uses-permission android:name="android.permission.RECORD_AUDIO" />
<uses-permission android:name="android.permission.MODIFY_AUDIO_SETTINGS" />

<uses-feature android:name="android.hardware.bluetooth_le" android:required="tr
```
```
```

1. REQUEST RUNTIME PERMISSIONS

Kotlin

```kotlin
import android.Manifest
import androidx.core.app.ActivityCompat
import androidx.core.content.ContextCompat

fun requestHearingAidPermissions(activity: Activity) {
    val permissions = arrayOf(
        Manifest.permission.BLUETOOTH_SCAN,
        Manifest.permission.BLUETOOTH_CONNECT,
        Manifest.permission.RECORD_AUDIO,
        Manifest.permission.MODIFY_AUDIO_SETTINGS
    )

    val permissionsToRequest = permissions.filter {
        ContextCompat.checkSelfPermission(activity, it) != PackageManager.PERMI
    }.toTypedArray()

    if (permissionsToRequest.isNotEmpty()) {
        ActivityCompat.requestPermissions(activity, permissionsToRequest, 1001)
    }
}
```

1. DISCOVER HEARING AIDS

Kotlin
```

```
import android.bluetooth.BluetoothManager
import android.bluetooth.BluetoothAdapter
import android.bluetooth.le.ScanCallback
import android.bluetooth.le.ScanResult

class HearingAidScanner(private val context: Context) {
    private val bluetoothManager = context.getSystemService(Context.BLUETOOTH_S
    private val bluetoothAdapter = bluetoothManager.adapter
    private val bluetoothScanner = bluetoothAdapter?.bluetoothLeScanner

    private val discoveredHearingAids = mutableListOf<BluetoothDevice>()

    fun startScanning() {
        val scanCallback = object : ScanCallback() {
            override fun onScanResult(callbackType: Int, result: ScanResult?) {
                super.onScanResult(callbackType, result)

                result?.device?.let { device ->
                    if (isHearingAidDevice(device)) {
                        discoveredHearingAids.add(device)
                        println("Found hearing aid: ${device.name} - Signal: ${
                    }
                }
            }

            override fun onScanFailed(errorCode: Int) {
                super.onScanFailed(errorCode)
                println("Scan failed with error code: $errorCode")
            }
        }

        bluetoothScanner?.startScan(scanCallback)
    }

    private fun isHearingAidDevice(device: BluetoothDevice): Boolean {
        // Check device class and name for hearing aid indicators
        val deviceClass = device.bluetoothClass?.deviceClass ?: 0
        val deviceName = device.name ?: ""

        // Hearing aids typically have specific class codes
        val hearingAidClass = 0x0000 // Hearing aid class code

        return deviceClass == hearingAidClass ||
               deviceName.contains("ReSound", ignoreCase = true) ||
               deviceName.contains("Phonak", ignoreCase = true) ||
               deviceName.contains("Widex", ignoreCase = true) ||
               deviceName.contains("Signia", ignoreCase = true) ||
```

```
                deviceName.contains("Oticon", ignoreCase = true)
    }
}
```

1. CONNECT TO HEARING AID

Kotlin

```kotlin
import android.bluetooth.BluetoothGatt
import android.bluetooth.BluetoothGattCallback
import android.bluetooth.BluetoothProfile

class HearingAidConnector(private val context: Context) {
    private var bluetoothGatt: BluetoothGatt? = null

    fun connectToHearingAid(device: BluetoothDevice) {
        val gattCallback = object : BluetoothGattCallback() {
            override fun onConnectionStateChange(gatt: BluetoothGatt?, status:
                super.onConnectionStateChange(gatt, status, newState)

                if (newState == BluetoothProfile.STATE_CONNECTED) {
                    println("Connected to hearing aid: ${gatt?.device?.name}")
                    gatt?.discoverServices()
                } else if (newState == BluetoothProfile.STATE_DISCONNECTED) {
                    println("Disconnected from hearing aid")
                }
            }

            override fun onServicesDiscovered(gatt: BluetoothGatt?, status: Int
                super.onServicesDiscovered(gatt, status)

                if (status == BluetoothGatt.GATT_SUCCESS) {
                    println("Services discovered")
                    // Configure audio routing
                    configureAudioRouting(gatt)
                }
            }
        }

        bluetoothGatt = device.connectGatt(context, false, gattCallback)
    }

    private fun configureAudioRouting(gatt: BluetoothGatt?) {
        val audioManager = context.getSystemService(Context.AUDIO_SERVICE) as A

        // Route audio to hearing aid
        audioManager.setBluetoothScoOn(true)
        audioManager.startBluetoothSco()
```

```
    }
}
```

1. CONFIGURE AUDIO SESSION

Kotlin

```kotlin
import android.media.AudioManager
import android.media.AudioAttributes
import android.media.AudioFocusRequest

fun configureAudioSession(context: Context) {
    val audioManager = context.getSystemService(Context.AUDIO_SERVICE) as Audio

    // Request audio focus
    val audioAttributes = AudioAttributes.Builder()
        .setUsage(AudioAttributes.USAGE_VOICE_COMMUNICATION)
        .setContentType(AudioAttributes.CONTENT_TYPE_SPEECH)
        .build()

    val focusRequest = AudioFocusRequest.Builder(AudioManager.AUDIOFOCUS_GAIN)
        .setAudioAttributes(audioAttributes)
        .setOnAudioFocusChangeListener { focusChange ->
            when (focusChange) {
                AudioManager.AUDIOFOCUS_GAIN -> println("Audio focus gained")
                AudioManager.AUDIOFOCUS_LOSS -> println("Audio focus lost")
            }
        }
        .build()

    audioManager.requestAudioFocus(focusRequest)

    // Set audio mode to communication
    audioManager.mode = AudioManager.MODE_IN_COMMUNICATION
}
```

1. IMPLEMENT VOLUME CONTROL

Kotlin

```kotlin
class HearingAidVolumeController(private val context: Context) {
    private val audioManager = context.getSystemService(Context.AUDIO_SERVICE)

    fun setPhoneVolume(volume: Int) {
        audioManager.setStreamVolume(
            AudioManager.STREAM_VOICE_CALL,
            volume,
```

```
            AudioManager.FLAG_SHOW_UI
        )
    }

    fun setHearingAidVolume(volume: Int) {
        // Send volume command to hearing aid via Bluetooth
        // Implementation depends on specific hearing aid model
    }

    fun getPhoneVolume(): Int {
        return audioManager.getStreamVolume(AudioManager.STREAM_VOICE_CALL)
    }

    fun getMaxVolume(): Int {
        return audioManager.getStreamMaxVolume(AudioManager.STREAM_VOICE_CALL)
    }
}
```

1. HANDLE AUDIO FOCUS

Kotlin

```
fun handleAudioFocus(context: Context) {
    val audioManager = context.getSystemService(Context.AUDIO_SERVICE) as Audio

    val focusListener = AudioManager.OnAudioFocusChangeListener { focusChange -
        when (focusChange) {
            AudioManager.AUDIOFOCUS_GAIN -> {
                // Resume playback
                println("Audio focus gained, resuming playback")
            }
            AudioManager.AUDIOFOCUS_LOSS -> {
                // Stop playback
                println("Audio focus lost, stopping playback")
            }
            AudioManager.AUDIOFOCUS_LOSS_TRANSIENT -> {
                // Pause playback
                println("Audio focus lost temporarily, pausing playback")
            }
        }
    }

    val audioAttributes = AudioAttributes.Builder()
        .setUsage(AudioAttributes.USAGE_VOICE_COMMUNICATION)
        .build()

    val focusRequest = AudioFocusRequest.Builder(AudioManager.AUDIOFOCUS_GAIN)
        .setAudioAttributes(audioAttributes)
```

```
        .setOnAudioFocusChangeListener(focusListener)
        .build()

    audioManager.requestAudioFocus(focusRequest)
}
```

1. TEST CONNECTION

Kotlin

```
fun testHearingAidConnection(context: Context) {
    val audioManager = context.getSystemService(Context.AUDIO_SERVICE) as Audio

    // Generate test tone (440 Hz, 1 second)
    val sampleRate = 44100
    val duration = 1 // seconds
    val frequency = 440.0 // Hz
    val amplitude = 0.3 // 30% volume

    val numSamples = sampleRate * duration
    val samples = ShortArray(numSamples)

    for (i in 0 until numSamples) {
        val phase = 2.0 * Math.PI * frequency * i / sampleRate
        samples[i] = (amplitude * 32767 * Math.sin(phase)).toInt().toShort()
    }

    // Play test tone
    val audioTrack = android.media.AudioTrack(
        AudioManager.STREAM_VOICE_CALL,
        sampleRate,
        android.media.AudioFormat.CHANNEL_OUT_MONO,
        android.media.AudioFormat.ENCODING_PCM_16BIT,
        samples.size * 2,
        android.media.AudioTrack.MODE_STATIC
    )

    audioTrack.write(samples, 0, samples.size)
    audioTrack.play()

    println("Test tone sent to hearing aid")
}
```

---

4. User Interface for Hearing Aid Settings

Settings Screen Flow

Plain Text

```
HEARING AID SETTINGS SCREEN:

┌─────────────────────────────────────────────┐
│ ⚙️ HEARING AID SETTINGS                      │
├─────────────────────────────────────────────┤
│                                             │
│ 🔗 HEARING AID CONNECTION                    │
│ ├─ Status: Connected ✅                      │
│ ├─ Device: ReSound Linx Quattro             │
│ ├─ Signal Strength: ████▒▒▒▒ 75%            │
│ └─ [Disconnect] [Reconnect]                 │
│                                             │
│ 🎵 AUDIO ROUTING                            │
│ ├─ Route to: Hearing Aid ✓                  │
│ ├─ Also use phone speaker: ☐                │
│ └─ [Test Audio]                             │
│                                             │
│ 🔊 VOLUME CONTROL                           │
│ ├─ Phone Volume: ████▒▒▒▒ 60%               │
│ ├─ Hearing Aid Volume: ██████▒▒ 70%         │
│ └─ Independent Control: ✓                   │
│                                             │
│ 🔄 FEEDBACK REDUCTION                       │
│ ├─ Enable Feedback Reduction: ✓             │
│ ├─ Echo Cancellation: ✓                     │
│ └─ Noise Reduction: ✓                       │
│                                             │
│ 🎧 HEARING AID MODEL                        │
│ ├─ Brand: ReSound                           │
│ ├─ Model: Linx Quattro                      │
│ ├─ Firmware: 2.5.1                          │
│ └─ [Update Firmware]                        │
│                                             │
│ 📱 DEVICE PAIRING                           │
│ ├─ iOS Device: iPhone 14 Pro                │
│ ├─ Android Device: Samsung Galaxy S23       │
│ └─ [Add Device]                             │
│                                             │
│ ⚡ BATTERY STATUS                           │
│ ├─ Left Hearing Aid: 85%                    │
│ ├─ Right Hearing Aid: 82%                   │
│ └─ [Low Battery Alert: ON]                  │
│                                             │
│ 🧪 TEST CONNECTION                          │
```

```
│ └─ [Play Test Tone] [Test Microphone]   │
│                                         │
└─────────────────────────────────────────┘


PAIRING FLOW:
1. User taps "Connect Hearing Aid"
2. App scans for nearby hearing aids
3. Displays list of found devices
4. User selects their hearing aid
5. App pairs via Bluetooth
6. Tests connection with tone
7. Confirms successful pairing
8. Shows connection status
```

---

5. API Endpoints for Hearing Aid Management

Backend API

Plain Text

```
GET /api/hearing-aid/status
Request: { user_id: "senior_12345" }
Response: {
  connected: true,
  device_name: "ReSound Linx Quattro",
  device_brand: "ReSound",
  device_model: "Quattro",
  signal_strength: 75,
  battery_left: 85,
  battery_right: 82,
  audio_routing: "hearing_aid",
  phone_volume: 60,
  hearing_aid_volume: 70,
  feedback_reduction: true,
  echo_cancellation: true,
  noise_reduction: true,
  last_connected: "2025-03-20T10:30:00Z"
}

POST /api/hearing-aid/connect
Request: {
  user_id: "senior_12345",
  device_id: "hearing_aid_xyz",
  device_name: "ReSound Linx Quattro",
  device_brand: "ReSound"
```

```
}
Response: {
  success: true,
  connected: true,
  device_id: "hearing_aid_xyz",
  connection_time: "2025-03-20T10:35:00Z"
}

POST /api/hearing-aid/disconnect
Request: { user_id: "senior_12345" }
Response: {
  success: true,
  disconnected: true,
  disconnection_time: "2025-03-20T10:40:00Z"
}

PUT /api/hearing-aid/settings
Request: {
  user_id: "senior_12345",
  audio_routing: "hearing_aid" | "phone_speaker" | "both",
  phone_volume: 60,
  hearing_aid_volume: 70,
  feedback_reduction: true,
  echo_cancellation: true,
  noise_reduction: true
}
Response: {
  success: true,
  settings_updated: true,
  updated_at: "2025-03-20T10:45:00Z"
}

POST /api/hearing-aid/test-connection
Request: { user_id: "senior_12345" }
Response: {
  success: true,
  test_tone_sent: true,
  signal_strength: 75,
  audio_quality: "excellent" | "good" | "fair" | "poor"
}

GET /api/hearing-aid/supported-brands
Request: {}
Response: {
  brands: [
    {
      name: "ReSound",
      models: ["Linx", "Quattro", "Omnia", "One"],
```

```
      ios_support: true,
      android_support: true
    },
    {
      name: "Phonak",
      models: ["Audeo", "Bolero", "Naida", "Virto"],
      ios_support: true,
      android_support: true
    },
    // ... more brands
  ]
}

POST /api/hearing-aid/battery-alert
Request: {
  user_id: "senior_12345",
  battery_level: 15,
  side: "left" | "right"
}
Response: {
  success: true,
  alert_sent: true,
  family_notified: true
}
```

---

6. Database Schema for Hearing Aid Data

Tables

Plain Text

```
TABLE: user_hearing_aids
- hearing_aid_id (PK)
- user_id (FK)
- device_name (text)
- device_brand (text)
- device_model (text)
- device_id (text)
- ios_device_id (text)
- android_device_id (text)
- is_connected (boolean)
- signal_strength (integer 0-100)
- battery_left (integer 0-100)
- battery_right (integer 0-100)
- last_connected_at (timestamp)
```

```
- created_at (timestamp)
- updated_at (timestamp)

TABLE: hearing_aid_settings
- setting_id (PK)
- user_id (FK)
- hearing_aid_id (FK)
- audio_routing (enum: hearing_aid, phone_speaker, both)
- phone_volume (integer 0-100)
- hearing_aid_volume (integer 0-100)
- feedback_reduction_enabled (boolean)
- echo_cancellation_enabled (boolean)
- noise_reduction_enabled (boolean)
- low_battery_alert_enabled (boolean)
- low_battery_threshold (integer 0-100)
- created_at (timestamp)
- updated_at (timestamp)

TABLE: hearing_aid_connection_logs
- log_id (PK)
- user_id (FK)
- hearing_aid_id (FK)
- event_type (enum: connected, disconnected, battery_low, signal_weak, test_ton
- details (JSON)
- timestamp (timestamp)

TABLE: hearing_aid_battery_alerts
- alert_id (PK)
- user_id (FK)
- hearing_aid_id (FK)
- side (enum: left, right, both)
- battery_level (integer)
- family_notified (boolean)
- created_at (timestamp)
```

---

7. Success Metrics for Hearing Aid Integration

Plain Text

```
ADOPTION METRICS:
- % of users with hearing loss who enable hearing aid support: Target 40%+
- % of hearing aid users who complete pairing: Target 85%+
- Average time to complete pairing: Target <5 minutes

QUALITY METRICS:
- Audio quality rating (1-5): Target 4.5+
```

```
- Connection stability (uptime): Target 99%+
- Feedback/echo complaints: Target <2%

ENGAGEMENT METRICS:
- Daily active users with hearing aid connected: Target 60%+
- Average session duration with hearing aid: Target 15+ minutes
- User satisfaction with hearing aid integration: Target 4.5/5

ACCESSIBILITY METRICS:
- % of seniors 70+ with hearing loss using app: Target 10%+
- Hearing aid brand coverage: Target 85%+ of market
- Platform coverage: Target iOS 95%+, Android 90%+
```

---

8. Implementation Checklist for Replit

Plain Text

```
BEFORE BUILDING:
☐ Confirm supported hearing aid brands (8 major brands)
☐ Confirm iOS MFi certification requirements
☐ Confirm Android ASHA API requirements
☐ Confirm audio routing strategy (hearing aid, phone, both)

DURING BUILD:
☐ iOS implementation (CoreBluetooth, AVAudioSession)
☐ Android implementation (BluetoothManager, AudioManager)
☐ Settings UI (connection, volume, feedback reduction)
☐ API endpoints (connect, disconnect, settings, test)
☐ Database tables (hearing aids, settings, logs, alerts)
☐ Battery monitoring and alerts
☐ Audio quality testing

AFTER BUILD:
☐ Test with real hearing aids (if available)
☐ Test on iOS 12.2+ devices
☐ Test on Android 10+ devices
☐ Test audio quality and latency
☐ Test feedback reduction and echo cancellation
☐ Test battery monitoring
☐ Gather user feedback
☐ Iterate based on feedback
```

Summary

This hearing aid connectivity specification enables SeniorShield to serve the 30 million seniors with hearing loss in the US. By supporting 8 major hearing aid brands and providing seamless audio routing, volume control, and feedback reduction, SeniorShield becomes the **most accessible voice assistant for seniors**.

Key differentiators:

- ✅ Direct hearing aid integration (not just phone speaker)
- ✅ Support for 8 major brands (85% market coverage)
- ✅ Independent volume control (phone + hearing aid)
- ✅ Feedback reduction and echo cancellation
- ✅ Battery monitoring and alerts
- ✅ Seamless iOS and Android support

This positions SeniorShield as the **most accessible and user-friendly voice assistant for seniors with hearing loss**.